ЗВІТ З ПРАКТИЧНОЇ РОБОТИ №1

«Діагностика кодової бази»

Предмет: Комплексне перепроєктування та удосконалення програмного забезпечення

Варіант №6 — Express.js (JavaScript)

Виконав: Шаровський Владислав Сергійович

Група: ІПЗ-21-1 ict

Спеціальність: Інженерія програмного забезпечення

Проєкт: https://github.com/expressjs/express

Область аналізу: 6 файлів каталогу lib/

Дата вимірювання: 07.11.2026

Звіт оновлено за результатами локального запуску AST-аналізатора та перевірки ESLint. Результати ESLint надано 09.11.2026.

1. Мета та область аналізу

Мета роботи — дослідити якість наявної кодової бази засобами статичного аналізу: визначити розмір, складність, залежності, потенційні проблемні зони та оцінити технічний борг.

Об’єкт дослідження — Express.js, вебфреймворк для Node.js. Аналіз обмежено шістьма файлами каталогу lib/, без залежностей із node_modules. Такий обсяг відповідає вимозі 2000–5000 рядків.

Файл	Фізичні рядки	Функцій	Роль
application.js	632	25	Конфігурація і маршрути
express.js	82	2	Створення застосунку
request.js	528	20	HTTP-запит
response.js	1051	41	HTTP-відповідь
utils.js	272	14	Утиліти
view.js	206	7	Шаблони і подання

Разом: 2771 фізичний рядок, 109 функцій. Частина рядків є порожніми, коментарями та службовими конструкціями; цей показник не є чистим LOC.

1.1. Використані засоби

Інструменти: Visual Studio Code, Node.js, acorn/acorn-walk (аналіз AST), ESLint (перевірка правил), власні скрипти tools/audit.mjs та tools/lint.mjs. Джерела вимірювань: results/metrics.json (07.11.2026), results/eslint.json (09.11.2026).

Досліджена копія: C:\Users\vlads\VSProjects\express\lib. SHA коміту не записаний в metrics.json; перед остаточною здачею його треба додати для відтворюваності.

2. Фактичні результати статичного аналізу

Локальний аналізатор сформував metrics.json, metrics.html і top_methods.csv. Дані отримані з AST; нижче наведено лише перевірені показники саме цих файлів.

Метрика	Значення	Пояснення
Проаналізовано файлів	6	У каталозі lib/
Фізичних рядків	2771	Сума рядків 6 файлів
Функцій	109	Функції, знайдені AST
Функцій з M > 10	2	Підвищена складність
Функцій з M > 20	1	Висока складність
Точних міжфайлових збігів, вікно 6 рядків	0	За вузьким правилом скрипта
ESLint-порушень	Не визначено	Файл eslint.json відсутній

2.1. Обмеження методики

Власний скрипт рахує умовні переходи за навчальним правилом і не є повною заміною SonarQube. Відсутність точних міжфайлових збігів із вікном 6 рядків не означає відсутності логічного дублювання. Відсутність eslint.json не дозволяє назвати кількість стилістичних порушень або потенційних дефектів.

2.2. Інтерпретація

Найбільша концентрація складної логіки припадає на response.js. Необхідна насамперед перевірка поведінки формування відповіді та допоміжних сценаріїв передачі файлів; самі по собі високі метрики ще не свідчать про дефект.

3. Цикломатична складність

Цикломатична складність M оцінює кількість незалежних шляхів виконання. За методичними рекомендаціями M до 10 — низький ризик, 11–20 — помірний, понад 20 — високий. Наведено 20 найскладніших функцій із metrics.json.

№	Файл	Функція (рядок)	M	Ризик
1	response.js	send (126)	23	високий
2	response.js	download (437)	13	помірний
3	application.js	render (522)	10	низький
4	response.js	cookie (748)	8	низький
5	view.js	View (52)	8	низький
6	request.js	(anonymous) (469)	7	низький
7	response.js	jsonp (264)	7	низький
8	response.js	sendFile (375)	7	низький
9	response.js	(anonymous) (408)	7	низький
10	utils.js	acceptParams (89)	7	низький
11	application.js	use (190)	6	низький
12	request.js	header (64)	6	низький
13	response.js	header (671)	6	низький
14	response.js	redirect (815)	6	низький
15	utils.js	compileETag (130)	6	низький
16	utils.js	compileQueryParser (162)	6	низький
17	utils.js	compileTrust (194)	6	низький
18	application.js	set (351)	5	низький
19	request.js	protocol (297)	5	низький
20	request.js	host (418)	5	низький

Дві функції перевищують поріг M > 10: response.js → send (M = 23) та download (M = 13). Інші 18 наведених функцій мають M від 5 до 10.

4. Пояснення розрахунку складності

Скрипт використовує AST-правило: M = 1 + кількість вузлів рішень; вкладені функції розглядаються окремо. Результат залежить від набору синтаксичних вузлів, які враховує аналізатор. Це важливо під час зіставлення з іншими інструментами.

Приклад із реального звіту	Значення M	Для ручної перевірки
response.js → send, рядки 126–222	23	Відкрити вказаний діапазон та порахувати розгалуження
response.js → download, рядки 437–486	13	Відкрити вказаний діапазон та порахувати розгалуження
application.js → render, рядки 522–575	10	Відкрити вказаний діапазон та порахувати розгалуження
response.js → cookie, рядки 748–781	8	Відкрити вказаний діапазон та порахувати розгалуження
view.js → View, рядки 52–95	8	Відкрити вказаний діапазон та порахувати розгалуження

Вимога методички щодо ручної перевірки цикломатичної складності 3–5 функцій залишається окремим етапом: значення M отримано автоматично, але незалежний підрахунок вузлів рішень у вихідному коді не задокументований. Наведені показники не слід називати вручну перевіреними.

4.1. Як перевірити вручну

У VS Code відкрити відповідний файл Express.js і перейти до рядка, наведеного у таблиці. Підрахувати базову одиницю та гілки if, цикли, умовні оператори, case і логічні && / || — саме відповідно до правил навчального аналізатора. Занести окремі кількості в таблицю та пояснити розбіжності, якщо вони є.

Практичний пріоритет для перевірки: send, download, render, cookie, View. Перші дві — єдині з підвищеним ризиком за метрикою M.

5. Зв’язність і зчеплення модулів

Зв’язність (cohesion) — наскільки функції виконують спільне завдання. Зчеплення (coupling) — кількість прямих залежностей. У цьому звіті зв’язність визначено якісно за призначенням модулів; залежності пораховано за статичними require-викликами.

Модуль	Залежностей →	Локальних ←	Зв’язність
application.js	11	1	середня
express.js	8	1	висока
request.js	8	1	висока
response.js	16	1	середня
utils.js	8	2	середня
view.js	3	1	висока

5.1. Обґрунтування

application.js: Налаштування, маршрутизація та рендеринг в одному модулі. Статично виявлено 11 прямих залежностей, з інших розглянутих модулів — 1.

express.js: Створення та компонування застосунку. Статично виявлено 8 прямих залежностей, з інших розглянутих модулів — 1.

request.js: Операції читання і властивості HTTP-запиту. Статично виявлено 8 прямих залежностей, з інших розглянутих модулів — 1.

response.js: Об’єднує надсилання, файли, cookies, перенаправлення. Статично виявлено 16 прямих залежностей, з інших розглянутих модулів — 1.

utils.js: Службові функції різного призначення. Статично виявлено 8 прямих залежностей, з інших розглянутих модулів — 2.

view.js: Пошук і відображення шаблонів. Статично виявлено 3 прямих залежностей, з інших розглянутих модулів — 1.

Показник локальних входжень враховує лише вибрані 6 файлів, тому не є повною оцінкою впливу модуля на весь Express.js. Високий ризик «God Object» тільки з цих даних не доведений.

6. Проблемні зони й візуалізація

Для пріоритизації використано максимальну M та обсяг файлів. Це ознаки складності супроводу, а не автоматичні докази помилок.

Пріоритет	Файл	Макс. M	Рядків	Чому важливий
1	response.js	23	1051	Є функції M > 10
2	application.js	10	632	Потребує огляду за розміром / роллю
3	view.js	8	206	Потребує огляду за розміром / роллю
4	request.js	7	528	Потребує огляду за розміром / роллю
5	utils.js	7	272	Потребує огляду за розміром / роллю



Рисунок 1 — Максимальна цикломатична складність за файлами (побудовано за metrics.json).

7. Оцінка технічного боргу (SQALE)

Метод SQALE у навчальному спрощенні оцінює час усунення виявлених проблем. За методичними рекомендаціями для порушень стилю — 5 хв, для M > 20 — 120 хв, для дублювання — 30 хв, для потенційного дефекту — 45 хв.

Категорія	Кількість	Час/випадок	Разом
Складність M > 20	1	120 хв	120 хв
Порушення ESLint: no-case-declarations, no-useless-escape, no-unused-vars	3	5 хв	15 хв
ESLint: no-redeclare — потрібна перевірка впливу	1	45 хв	45 хв
Точні міжфайлові дублікати (навчальний тест)	0	30 хв	0 хв

За навчальною моделлю: 1 метод із M > 20 × 120 хв = 120 хв. ESLint виявив 3 порушення, яким умовно призначено по 5 хв, та 1 повторне оголошення змінної — 45 хв на аналіз і безпечне виправлення. Разом з ESLint: 60 хв.

Оцінений Remediation Cost = 120 + 60 + 0 = 180 хв = 3 год. Нуль для дублювання означає лише відсутність точних міжфайлових збігів у простому тесті з вікном 6 рядків, а не доведену відсутність будь-якого дублювання. Інші типи проблем поза діапазоном правил ESLint можуть залишатися невиявленими.

7.1. Розрахунковий приклад, не фінальний рейтинг

Припущення для Development Cost: 10 фізичних рядків на 1 людино-годину. Development Cost = 2771 / 10 = 277,1 год. Technical Debt Ratio = (3 / 277,1) × 100% ≈ 1,08%. Це орієнтовна оцінка A (до 5%) за шкалою методички, а не рейтинг SonarQube.

Час усунення та продуктивність є оціночними припущеннями, а не замірами розробки. ESLint-правила налаштовані для навчального аналізу, тому TDR залежить від обраного набору правил і не охоплює всі можливі дефекти.

8. Висновки та рекомендації

На основі локального AST-аналізу шести файлів Express.js отримано 2771 фізичних рядків та 109 функцій. Дві функції мають M > 10, з них send — M = 23. ESLint додатково виявив 4 повідомлення (3 помилки, 1 попередження). Головні зони уваги — response.js і utils.js.

Структура проєкту загалом розділяє функції застосунку, запиту, відповіді, утиліт і шаблонів. Модуль response.js поєднує кілька видів операцій HTTP-відповіді, але твердження про God Object на підставі наявних метрик було б передчасним.

За поточним набором правил ESLint виявлено 4 порушення (3 помилки, 1 попередження). Навчальна оцінка боргу — 3 год, TDR ≈ 1,08% (A). Для кращої відтворюваності варто додати SHA коміту Express.js та окремо вручну звірити складність 3–5 функцій. Рефакторинг не є обов'язковою частиною роботи №1.

8.1. Команди для відтворення

У терміналі VS Code, перебуваючи у папці IPZ_Practical_01:

git -C ../express rev-parse HEAD

npm.cmd run audit

npm.cmd run lint

npx.cmd eslint ../express/lib/application.js ../express/lib/express.js ../express/lib/request.js ../express/lib/response.js ../express/lib/utils.js ../express/lib/view.js -f json -o results/eslint.json

Якщо команда lint завершується з кодом 1, це може означати знайдені ESLint-порушення; перевірити наявність і вміст results/eslint.json. Не запускати автоматичні виправлення на вихідному проєкті без окремого рішення.

8.2. Використані джерела

1. Практична робота №1 «Діагностика кодової бази», завдання викладача.

2. Методичні рекомендації до практичної роботи №1.

3. Express.js: https://github.com/expressjs/express

4. Фактичні результати: results/metrics.json, results/top_methods.csv, results/metrics.html (07.11.2026), results/eslint.json (09.11.2026).

9. Деталізація результатів ESLint

ESLint перевірив шість зазначених файлів і за активною конфігурацією повернув 3 помилки та 1 попередження (усього 4 повідомлення). Це не означає, що в коді існують чотири підтверджені функціональні дефекти: ESLint також перевіряє якість і стиль написання.

Файл, рядок	Правило	Рівень	Інтерпретація
response.js:138	no-case-declarations	Помилка	Оголошення const без блоку {} у case.
response.js:290	no-useless-escape	Помилка	Зайве екранування символа у регулярному виразі.
utils.js:99	no-redeclare	Помилка	Повторне оголошення змінної colonIndex.

view.js:202	no-unused-vars	Попередження	Змінна e у catch не використовується.

Файли application.js, express.js і request.js не мали повідомлень ESLint за поточною конфігурацією. Розподіл за файлами: response.js — 2, utils.js — 1, view.js — 1. У звіті не стверджується, що ESLint довів наявність експлуатаційних вразливостей.

Пріоритет усунення: спочатку перевірити поведінку response.js → send (M = 23) і повторне оголошення у utils.js, далі виправити відносно прості повідомлення про структуру case, зайве екранування та невикористану змінну. У цій практичній зміни до стороннього репозиторію не вносились.

Для відтворення результату використати npm.cmd run lint у робочій папці, а також npm.cmd run audit. Показники складності й ESLint отримано окремими інструментами та вони не повинні трактуватися як однакова метрика.